EPIC-F — Content Reporting and Moderation Tools — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The sixth per-epic technical spec. Builds on the architecture spec's Section 4.2 (`community.reports`, `community.moderation_actions`), Section 4.4 (audit logging scope), and Section 7.2 (moderation tooling as role-gated routes in the same web app) — read those first. Also resolves two forward-references left open by earlier specs: EPIC-B's question (its spec, Section 13) about where a moderator-forced handle rename belongs, and EPIC-E's requirement (its spec, Section 9) for a `request_correction` action type. Both are addressed in Section 3 below.

Source of truth: `docs/askapeer-prd-v0.1.md`, Section 9.3–9.5 (zero-tolerance rule, moderation access to identity, inadvertent disclosure) and Section 10.4 (case-discussion priority reporting).

This epic is where the platform's two highest-stakes policies — zero-tolerance anonymity enforcement and the "no PHI" rule — actually get executed, not just stated. It's also where a real inconsistency between two already-written specs surfaces (Section 7) and needs flagging for the consolidated review, per your instruction to hold cross-epic conflicts until every spec is drafted.

Contents

---

1. Scope

In scope: reporting content (or a handle — Section 2) with a category, the moderation review queue and its ordering, the full set of moderation actions and their downstream effects on other epics' data, and the audited real-identity-viewing action.

Out of scope: the admin panel's UI/routing mechanics (already specified generically in the architecture spec, Section 7.2 — role-gated `/admin/*` routes in the same Next.js app); this epic only specifies the data and actions those routes operate on.

---

2. Data model

Reuses `community.reports` and `community.moderation_actions`, defined in the architecture spec, Section 4.2, with one addition: `target_type` **needs a** `handle` **value**, not just `post`/`comment`. The PRD's zero-tolerance rule (Section 9.3) explicitly covers "collusion to de-anonymise members on or off the platform" — a report about a pattern of behaviour or an off-platform incident may have no specific post or comment to attach to, only a handle. Restricting reports to content-only targets would leave no way to file exactly the kind of report the platform's most serious rule anticipates.

```
community.reports
  id, reporter_handle_id, target_type enum(post, comment, handle),
  target_id, category, priority boolean generated from category,
  status enum(open, actioned, dismissed), created_at

community.moderation_actions           -- immutable: INSERT-only
grant
  id, report_id nullable, target_handle_id,
  action_type enum(remove_content, warn, suspend, expel,
                   request_correction, rename_handle),
  moderator_id, reason, created_at
```

`action_type` gains **two values beyond the architecture spec's original four**: `request_correction` (resolves EPIC-E's Section 9 dependency — the "request a corrected resubmission" action PRD Section 10.4 names for case discussions) and `rename_handle` (resolves EPIC-B's Section 13 open question — a moderator-forced handle rename, when a handle name itself turns out to be identifying or impersonating, is a moderation action in substance and belongs in this enum rather than as a bespoke EPIC-B-only endpoint). Both additions are proposed here as the natural home for these actions, since this spec is where the action-type vocabulary is actually owned — but see Section 10 for confirmation status.

---

3. Moderation action types

action_type	Effect	Owning epic for the effect
remove_content	community.posts/comments.status = removed	EPIC-C
warn	No status change; a logged, immutable record a member received a formal warning	This epic
suspend	community.handles.status = suspended; see Section 7 for whether this should also touch identity.members	EPIC-B (status)
expel	community.handles.status = expelled **and** identity.members.verification_status = expelled, both in the same transaction; permanent	EPIC-B (status), Section 7 (identity linkage — resolved)
request_correction	Case discussion returns to an editable/ unpublished state for resubmission	EPIC-E
rename_handle	community.handles.handle_name changed, old name recorded in community.handle_name_history	EPIC-B

Every action writes one community.moderation_actions row regardless of type — this is the immutable trail the architecture spec's Section 4.4 requires for "moderation actions" generally, not just the four original action types.

---

4. Report categories and queue ordering

The PRD names one specific category explicitly — identifiable_patient_information (Section 10.4), which "trigger[s] a priority review flag and are actioned before other report types." It does not enumerate a full category list. This spec proposes the following, since a report form needs *some* fixed set of categories to be usable, but the full list is this spec's own construction, not a PRD requirement:

- identifiable_patient_information — **priority** (explicit PRD requirement)
- anonymity_violation — attempting to reveal one's own or another's identity (PRD Section 9.3/9.5). **This spec proposes priority status for this category too**, even though the PRD only explicitly names patient-information reports as priority — the zero-tolerance rule around anonymity is described in equally or more severe terms ("immediate and permanent expulsion… no exceptions") than the patient-privacy policy, and it would be an odd asymmetry for the platform's other founding guarantee to sit in the ordinary queue. Flagged for explicit confirmation in Section 10 rather than assumed.
- harassment
- spam

- `other`

Queue ordering: priority categories (`identifiable_patient_information`, `anonymity_violation`) first, then by report age within each tier — the same pattern the EPIC-A spec already established for its own (structurally separate) verification queue, Section 6 there.

---

5. Viewing real identity — the audited exception

The architecture spec (Section 7.2) already establishes that "viewing a member's real identity from the admin panel is a distinct, explicit action… not implicit in viewing a report." This epic is where that action is actually triggered from:

```
POST /v1/admin/handles/:handle_id/reveal-identity
  body: { reason_code: reported_violation | legal_request |
safety_escalation, reason_note }
  -> resolves handle_id -> member_id via IdentityService
  -> writes identity.identity_access_log (architecture spec,
Section 4.1)
  -> returns real identity fields to the moderator
```

A moderator reviewing a report (Section 4) does **not** automatically see the reported handle's real identity — the report queue itself operates entirely on `handle_id`s, consistent with every other community-facing surface in the platform. Only this explicit, separately-logged action crosses the boundary, exactly matching the architecture spec's stated design intent.

---

6. API endpoints

```
POST   /v1/reports                            { target_type,
target_id, category, comment }

--- admin-only, moderator-role JWT claim required ---

GET    /v1/admin/reports?status=open&cursor=...    -- priority
categories first (Section 4)
GET    /v1/admin/reports/:report_id
POST   /v1/admin/reports/:report_id/action  { action_type,
target_handle_id, reason, ...type-specific-fields }
POST   /v1/admin/handles/:handle_id/reveal-identity   -- Section 5
```

`POST .../action`'s type-specific fields: `remove_content` needs the specific `post_id`/`comment_id`; `rename_handle` needs `new_handle_name`; `request_correction` needs the case-discussion `post_id`. A single endpoint with a discriminated body (rather than one endpoint per action type) keeps the audit-logging code path in one place, which matters more here than API surface elegance given every branch must reliably write to an immutable log.

---

7. The expulsion/re-registration gap — resolved

This section originally flagged a real gap spanning three specs (EPIC-A, EPIC-B, and this one); Adrian resolved it on 2026-07-14 and it's kept here, marked resolved, for the history rather than deleted:

- `identity.members.verification_status` (architecture spec, Section 4.1) was `pending | needs_more_info | approved_verified | rejected | suspended` — no `expelled` value existed.
- `community.handles.status` (Section 4.2) is `active | suspended | expelled` — no `rejected`/`needs_more_info` value, and its `suspended` isn't necessarily the same event as the identity-side `suspended` (that question — whether this epic's `suspend` action should also touch `identity.members` — remains genuinely open; see Section 10).
- **The consequence**: when this epic's `expel` action set `community.handles.status = expelled`, nothing updated `identity.members.verification_status`, so a permanently expelled person's registration was still sitting at `approved_verified` — nothing stopped them re-registering with the same credentials.

Decision: `identity.members.verification_status` gains an `expelled` value (architecture spec amended, Section 4.1). This epic's `expel` action now writes an `identity.verification_decisions` transition to `expelled` in the same transaction as the `community.handles` update (Section 3's table, above). EPIC-A's uniqueness constraint already blocks any non-`rejected` status by construction (its spec, Section 2), so `expelled` is covered automatically — and per Adrian's direction, a blocked reapplication attempt is additionally logged to `identity.reapplication_attempts` and surfaced for admin review (EPIC-A's spec, Section 6), not just silently rejected. Full history in `docs/2026-07-14-technical-specs-open-questions.md`, Section 2.

Still open: whether `suspend` should similarly write to `identity.members` (see Section 10) — suspension's temporary/reversible nature makes this a less clear-cut case than expulsion's permanence.

---

8. Non-functional notes specific to EPIC-F

- **Immutable actions**: `community.moderation_actions` and `identity.identity_access_log` are both INSERT-only at the database-role level, per the architecture spec, Section 4.4 — this epic introduces no new tables needing that treatment, only new `action_type` values within the existing immutable table.
- **Response-time KPIs**: the PRD (Section 13) names "median moderation response time" and "PHI report resolution time — must be fast" as KPIs, without a numeric target. This epic's data model supports measuring both (`created_at` on the report, `created_at` on the resulting `moderation_actions` row), but no SLA is defined to build alerting against — flagged in Section 10.

---

9. Test plan

- **Priority ordering**: `identifiable_patient_information` and (pending confirmation) `anonymity_violation` reports surface before other categories regardless of submission order.
- **Handle-targeted reports**: a report with `target_type = handle` and no associated post/comment is valid and appears in the queue (covers the "off-platform collusion" case PRD Section 9.3 anticipates).
- **Identity reveal is separately logged**: viewing a report never itself produces an `identity_access_log` row; only `POST .../reveal-identity` does, and it requires a `reason_code`.
- **Action immutability**: no `UPDATE`/`DELETE` grant exists on `community.moderation_actions` at the database-role level (mirrors the EPIC-A spec's equivalent test for `identity.verification_decisions`).
- **New action types**: `request_correction` correctly unpublishes the target case discussion (coordinating with EPIC-E's flow); `rename_handle` writes `handle_name_history` and validates against the same blocklist/uniqueness rules as ordinary handle creation (EPIC-B, Section 3).
- `expel` **writes both statuses atomically**: `community.handles.status = expelled` and `identity.members.verification_status = expelled` both land, or neither does, on a forced failure of either write (same pattern as EPIC-A's Section 10 test for the verification state machine); a subsequent registration attempt using the expelled member's credentials is blocked and logged (EPIC-A spec, Section 10).

---

10. Open questions

- `anonymity_violation` **as a priority category** (Section 4): proposed here but not a PRD-stated requirement — needs explicit confirmation given the PRD only names patient-information reports as priority.
- ~~**The expulsion/re-registration gap**~~ — **resolved 2026-07-14**, see Section 7 above.
- **Should** `suspend` **also write to** `identity.members.verification_status` (Section 7): unlike `expel`, this is genuinely unresolved — a handle-level suspension (this epic's action) and the identity-level `suspended` status (EPIC-A's, covering lapsed registration) may or may not be intended as the same event. Needs a decision, though it's lower-stakes than the expulsion case since suspension is reversible either way.
- `rename_handle` **and** `request_correction` **as new action types**: proposed here as the natural resolution of two other specs' forward-references (Section 3) — worth a final check that EPIC-B and EPIC-E's specs should indeed be read as pointing here, rather than wanting their own bespoke handling.
- **No numeric moderation-response SLA** (Section 8): the PRD's KPIs name "must be fast" without a figure — worth a concrete target before building alerting/staffing plans around it.
- **Full report-category list** (Section 4): `harassment`, `spam`, `other` alongside the two priority categories is this spec's own proposal, not confirmed against the PRD (which doesn't enumerate a full list) — worth a sanity check with Andrew Renshaw given his domain familiarity with what reports are likely to actually look like in practice.